Lab Report – EE-170L Computer Programming (Lab 05)

NAME	saifullah
ID	725
SEMESTER	2nd
LAB 5	(programming)
DATE	10/3/2026

1. Objective

The main goal of this lab is to practice using different types of loops in C++. Loops are one of the most important concepts in programming because they allow us to repeat tasks without writing the same code again and again. In this lab, we focus on:

- **The while loop**
- **The nested while loop**
- **The do…while loop**

By the end of this lab, students should be able to:

- **Write programs that use loops to display sequences of numbers.**
- **Understand the difference between while and do…while loops.**
- **Apply loops to solve problems like summation.**

- **Recognize how nested loops can be used to print patterns.**

2. Theory

2.1 While Loop

The while loop is a control structure that repeats a block of code as long as a condition is true. The condition is checked before the loop body runs. If the condition is false at the start, the loop body will not run at all.

General Syntax:

```
initialization;
while (condition) {
   statements;
   update variable;
}
```

Key Points:

- Initialization sets the starting value.
- The condition decides whether the loop continues.

- The update changes the variable so the loop eventually stops.

2.2 Nested While Loop

A nested loop is a loop placed inside another loop. The outer loop usually controls how many times the entire structure runs (such as the number of rows), while the inner loop manages the repeated actions within each cycle (such as columns or repeated values). Nested loops are commonly used to create tables, grids, and different number patterns.

2.3 Do...While Loop

The do...while loop is similar to the while loop but with one difference: the body runs **at least once** before the condition is checked. This is useful when we want the program to ask for input or perform an action at least once, even if the condition is false later.

General Syntax:

```
initialization;
do {
   statements;
   update variable;
} while (condition);
```

3. Programs and Outputs

Task 1: Print numbers 1 to 10 using while loop

```
#include <iostream>
using namespace std;
int main() {
   int i = 1;
   while (i <= 10) {
      cout << i << endl;
      i++;
   }
   return 0;
}
```

Output:

1,2,3,4,5,6,7,8,910 in a column

Task 2: Repeat Task 1 using do...while loop

```
#include <iostream>
using namespace std;
int main() {
   int i = 1;
   do {
      cout << i << endl;
      i++;
   } while (i <= 10);
```

```
    return 0;
}
```

Output:

1 2 3 4 5 6 7 8 9 10 in a column

Task 3: Sum of numbers from 1 to n using while loop

```
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0, i = 1;
    cout << "Enter a positive integer: ";
    cin >> n;

    while (i <= n) {
        sum += i;
        i++;
    }

    cout << "Sum = " << sum << endl;
    return 0;
}
```

Output:

Enter a positive integer: 5

Sum = 15

4. Discussion

This lab showed how loops make programming easier and more efficient. Without loops, we would have to write many lines of code to print numbers or calculate sums. With loops, the same task can be done in just a few lines.

- The **while loop** is best when we don't know how many times the loop will run, but we have a condition to check.
- The **do...while loop** is useful when we want the loop to run at least once, such as asking for a password or user input.
- The **nested while loop** allows us to create patterns and handle multiple levels of repetition.

By practicing these loops, I learned how to control the flow of a program and how to avoid infinite loops by updating the loop variable correctly.

5. Conclusion

In this lab, I wrote programs using while, do…while, and nested loops. I learned how each loop works and where it can be used.

The while loop checks the condition before running.

The do…while loop runs at least once before checking the condition.

Nested loops help in printing patterns and repeating tasks inside another loop.

This lab improved my understanding of loops and helped me practice writing programs that repeat tasks automatically.